CS585 Problem Set 4 (Total points: 60 + 20 bonus)

Assignment adapted from Svetlana Lazebnik

Instructions

1. Assignment is due at **11.59 PM on Tuesday April 18 2023**.
2. Submission instructions:
   A. A single `.pdf` report that contains your work for Q1, Q2, Q3, and Q4 (optional bonus), Q5, Q6 (optional bonus), Q7 and Q8 (optional bonus). For Q5 and Q6, you should type out your responses in **LaTeX**. **Hand-written response is NOT allowed. If there's any scanned drawing, please insert it in the latex file.** If you are new to Latex, here's an online tool you can edit latex file. And a quick guide to write a piece of Latex, and how to add math in it. If your other parts of the report is not written in Latex, you can merge them with a .pdf merging tool, such as this.

   For Q1, Q2, Q3 and Q7, your response should also be electronic (**also no handwritten responses allowed**). Please refer to the **checklist** of at the end of this file to include the required result. **Points will not be given if the corresponding required item(s) in the checklist are missing in your report. (IMPORTANT!!!)** You will not receive credit for any results you have obtained, but failed to include directly in the PDF report file.

   PDF file should be submitted to Gradescope under `PS4`. Please tag the reponses in your PDF with the Gradescope questions outline as described in Submitting an Assignment.

   B. You also need to submit code for Q1, Q2, Q3 and Q7 in the form of a single `.py` file that includes all your code, all in the same directory. Code should be submitted to Gradescope under `PS4-Code`. ***Not submitting your code will lead to a loss of 100% of the points on Q1, Q2, Q3 and Q7.***

   C. ***We reserve the right to take off points for not following submission instructions.*** In particular, please tag the reponses in your PDF with the Gradescope questions outline as described in Submitting an Assignment.

Problems

1. **Camera Calibration [8 pts]**. For the pair of images in the folder `calibraion`, calculate the camera projection matrices by using 2D matches in both views and 3D point coordinates in `lab_3d.txt`. Once you have computed your projection matrices, you can evaluate them using the provided evaluation function (evaluate_points). The function outputs the projected 2-D points and residual error. Report the estimated 3 × 4 camera projection matrices (for each image), and residual error. **Hint:** The residual error should be < 20 and the squared distance of the projected 2D points from actual 2D points should be < 4.

In [1]:
```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

def evaluate_points(M, points_2d, points_3d):
    """
    Visualize the actual 2D points and the projected 2D points calculated fr
    the projection matrix
    You do not need to modify anything in this function, although you can if
    want to
    :param M: projection matrix 3 x 4
    :param points_2d: 2D points N x 2
    :param points_3d: 3D points N x 3
    :return:
    """
    N = len(points_3d)
    points_3d = np.hstack((points_3d, np.ones((N, 1))))
    points_3d_proj = np.dot(M, points_3d.T).T
    u = points_3d_proj[:, 0] / points_3d_proj[:, 2]
    v = points_3d_proj[:, 1] / points_3d_proj[:, 2]
    residual = np.sum(np.hypot(u-points_2d[:, 0], v-points_2d[:, 1]))
    points_3d_proj = np.hstack((u[:, np.newaxis], v[:, np.newaxis]))
    return points_3d_proj, residual

# Write your code here for camera calibration
def camera_calibration(pts_2d, pts_3d):
    """
    write your code to compute camera matrix
    """
    # <YOUR CODE>
    assert pts_2d.shape[0] == pts_3d.shape[0] # Make sure both numbers are "
    num_points = pts_3d.shape[0]

    # Make 3D points Homogeneous (append 1)
    pts_3d_homo = np.hstack( ( pts_3d, np.ones( ( num_points, 1)) ) ) # Shap

    # Make 2D Points Homogeneous
    # lambda * pts_2d = P * pts_3d_homo
    # pts_2d = P(lambda * P) * pts_3d_homo
    # Stack 6 of P to make A, since we need 12 parameter == 12 equations.
    # In this question, Lets stack all of them

    A = []
```

```
    for i in range(num_points):
        X, Y, Z, W = pts_3d_homo[i]
        u, v = pts_2d[i]

        # row 1 = [ 0^T, -X_{i}^T, v_{i}X_{i}^T} ]
        # row 2 = [ X^T, 0^T, -u_{i}X_{i}^T} ]

        # 0^T =  0, 0, 0, 0 | -X_{i}^T = -(X, Y, Z, 1) | v_{i}X_{i}^T} = v *
        row1 = [0, 0, 0, 0, -X, -Y, -Z, -1, v*X, v*Y, v*Z, v]

        # 0^T = X_{i}^T = X, Y, Z, 1  | 0, 0, 0, 0  | -u_{i}X_{i}^T} = -u *
        row2 = [X, Y, Z, 1, 0, 0, 0, 0, -u*X, -u*Y, -u*Z, -u]

        A.append(row1)
        A.append(row2)

    A = np.array(A)  # shape (2 * 20, 12)

    # Solve using SVD: A @ p = 0
    U, S, Vt = np.linalg.svd(A)
    p = Vt[-1]  # Solution is the last row of V^T (smallest singular value)
    P = p.reshape(3, 4)  # reshape to 3x4 projection matrix
    P = P / P[2][3] # set the last 3x4 as 1 to normalize

    return P

# Load 3D points, and their corresponding locations in
# the two images.
pts_3d = np.loadtxt('calibration/lab_3d.txt')
matches = np.loadtxt('calibration/lab_matches.txt')

# print lab camera projection matrices:
lab1_proj = camera_calibration(matches[:, :2], pts_3d)
lab2_proj = camera_calibration(matches[:, 2:], pts_3d)
print('lab 1 camera projection')
print(lab1_proj)

print('')
print('lab 2 camera projection')
print(lab2_proj)

# evaluate the residuals for both estimated cameras
_, lab1_res = evaluate_points(lab1_proj, matches[:, :2], pts_3d)
print('residuals between the observed 2D points and the projected 3D points:
print('residual in lab1:', lab1_res)
_, lab2_res = evaluate_points(lab2_proj, matches[:, 2:], pts_3d)
print('residual in lab2:', lab2_res)
```

```
lab 1 camera projection
[[-2.33260962e+00 -1.10025080e-01  3.37513233e-01  7.36686567e+02]
 [-2.31044166e-01 -4.79515070e-01  2.08722206e+00  1.53627263e+02]
 [-1.26377057e-03 -2.06774255e-03  5.14712341e-04  1.00000000e+00]]

lab 2 camera projection
[[-2.04586455e+00  1.18558243e+00  3.91381081e-01  2.44002874e+02]
 [-4.56804042e-01 -3.02392053e-01  2.14706068e+00  1.66030240e+02]
 [-2.24595257e-03 -1.09488059e-03  5.61389950e-04  1.00000000e+00]]
residuals between the observed 2D points and the projected 3D points:
residual in lab1: 13.545832896034767
residual in lab2: 15.544953452909517
```

2. **Camera Centers [3 pts].** Calculate the camera centers using the estimated or provided projection matrices. Report the 3D locations of both cameras in your report. **Hint:** Recall that the camera center is given by the null space of the camera matrix.

In [2]:
```
import scipy.linalg
# Write your code here for computing camera centers
def calc_camera_center(proj):
    """
    write your code to get camera center in the world
    from the projection matrix
    """
    # <YOUR CODE>
    null = scipy.linalg.null_space(proj) # null shape = 4 x 1
    c_homo = null[:, 0]
    c = c_homo / c_homo[3]
    return c


# compute the camera centers using
# the projection matrices
lab1_c = calc_camera_center(lab1_proj)
lab2_c = calc_camera_center(lab2_proj)
print('lab1 camera center', lab1_c)
print('lab2 camera center', lab2_c)
```

```
lab1 camera center [305.83276769 304.20103826  30.13699243   1.        ]
lab2 camera center [303.10003925 307.18428016  30.42166874   1.        ]
```

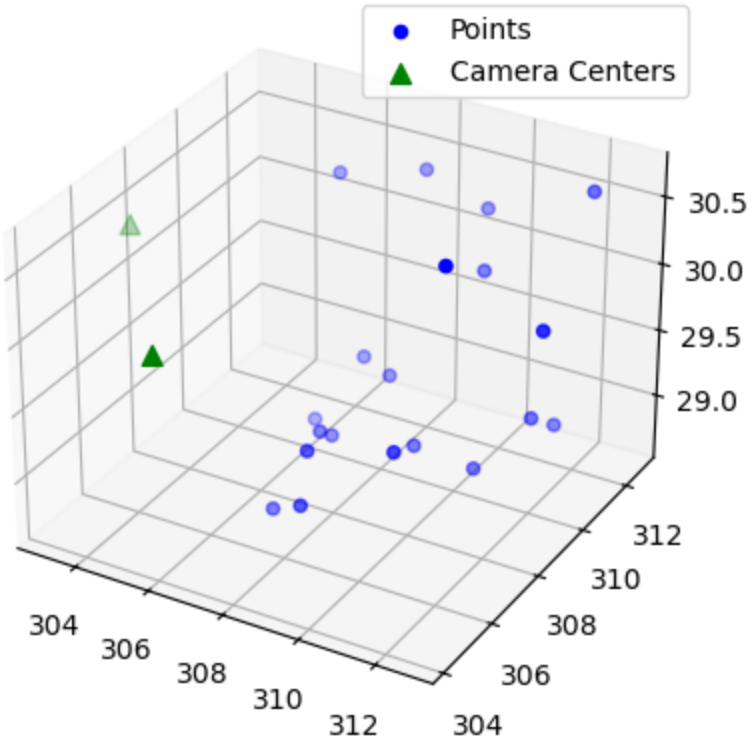
3. **Triangulation [8 pts].** Use linear least squares to triangulate the 3D position of each matching pair of 2D points using the two camera projection matrices. As a sanity check, your triangulated 3D points for the lab pair should match very closely the originally provided 3D points in `lab_3d.txt`. Display the two camera centers and reconstructed points in 3D. Include snapshots of this visualization in your report. Also report the residuals between the observed 2D points and the projected 3D points in the two images. Note: You do not need the camera centers to solve the triangulation problem. They are used just for the visualization.

In [3]:
```
# Write your code here for triangulation
from mpl_toolkits.mplot3d import Axes3D

def triangulation(lab_pt1, lab1_proj, lab_pt2, lab2_proj):
    """
    write your code to triangulate the points in 3D
    """
    # <YOUR CODE>
    num_points = lab_pt1.shape[0] # 20
    points_3d_lab = []

    for i in range(num_points):
        u1, v1 = lab_pt1[i]
        u2, v2 = lab_pt2[i]

        A = np.array([
            u1 * lab1_proj[2] - lab1_proj[0],
            v1 * lab1_proj[2] - lab1_proj[1],
            u2 * lab2_proj[2] - lab2_proj[0],
            v2 * lab2_proj[2] - lab2_proj[1]
        ])

        _, _, Vt = np.linalg.svd(A)
        X_homo = Vt[-1]
        X = X_homo[:3] / X_homo[3]
        points_3d_lab.append(X)

    return np.array(points_3d_lab)

def evaluate_points_3d(points_3d_lab, points_3d_gt):
    """
    write your code to evaluate the triangulated 3D points
    """
    # <YOUR CODE>
    residual = np.linalg.norm(points_3d_lab - points_3d_gt, axis=1)
    return residual


# triangulate the 3D point cloud for the lab data
matches_lab = np.loadtxt('calibration/lab_matches.txt')
lab_pt1 = matches_lab[:,:2]
lab_pt2 = matches_lab[:,2:]
points_3d_gt = np.loadtxt('calibration/lab_3d.txt')
points_3d_lab = triangulation(lab_pt1, lab1_proj, lab_pt2, lab2_proj) # Use
res_3d_lab = evaluate_points_3d(points_3d_lab, points_3d_gt)
print('Mean 3D reconstuction error for the lab data: ', round(np.mean(res_3d
_, res_2d_lab1 = evaluate_points(lab1_proj, lab_pt1, points_3d_lab)
_, res_2d_lab2 = evaluate_points(lab2_proj, lab_pt2, points_3d_lab)
print('2D reprojection error for the lab 1 data: ', np.mean(res_2d_lab1))
print('2D reprojection error for the lab 2 data: ', np.mean(res_2d_lab2))
# visualization of lab point cloud
camera_centers = np.vstack((lab1_c, lab2_c))
print(camera_centers.shape)
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
```

```
ax.scatter(points_3d_lab[:, 0], points_3d_lab[:, 1], points_3d_lab[:, 2], c=
ax.scatter(camera_centers[:, 0], camera_centers[:, 1], camera_centers[:, 2],
ax.legend(loc='best')
```

```
Mean 3D reconstuction error for the lab data:  0.01338
2D reprojection error for the lab 1 data:  6.391827886697235
2D reprojection error for the lab 2 data:  5.902480909114385
(2, 4)
```

Out[3]: <matplotlib.legend.Legend at 0x1246039d0>



4. **Extra Credits [3 pts].** Use the putative match generation and RANSAC code from `PS3` to estimate fundamental matrices without ground-truth matches. For this part, only use the normalized algorithm. Report the number of inliers and the average residual for the inliers. Compare the quality of the result with the one you get from ground-truth matches.

5. **Epipolar Geometry [15 pts total].** Let $M1$ and $M2$ be two camera matrices. We know that the fundamental matrix corresponding to these camera matrices is of the following form:

$$F = [a] \times A,$$

where $[a]\times$ is the matrix

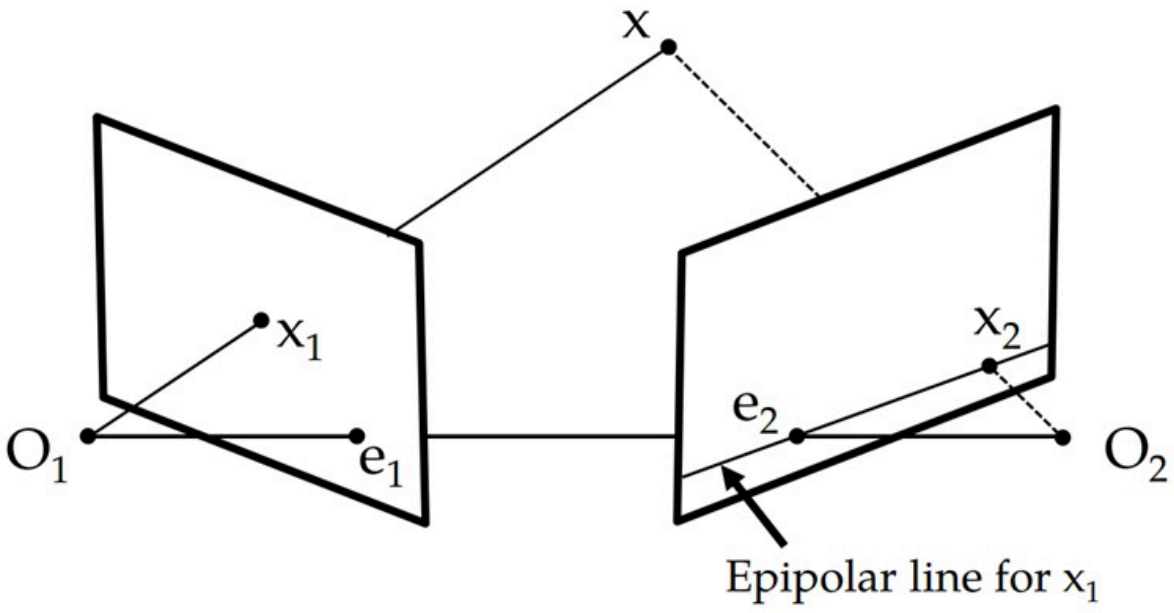
$$[a]\times = [\,0 \quad ay \quad -az - ay \quad 0 \quad axaz \quad -ax \quad 0\,].$$

Assume that $M1 = [I|0]$ and $M2 = [A|a]$, where A is a 3 × 3 (nonsingular) matrix.

6. **Combining with optical flow [5 pts]**. Propose an approach to modifies your optical flow implementation from Lab 5 to use epipolar geometry.

7. **Epipoles [10 pts]** Prove that the last column of $M2$, denoted by a, is one of the epipoles and draw your result in a diagram similar to the following image:

In [4]:
```
from PIL import Image
from IPython.display import display
display(Image.open('epipolar.jpg'))
```

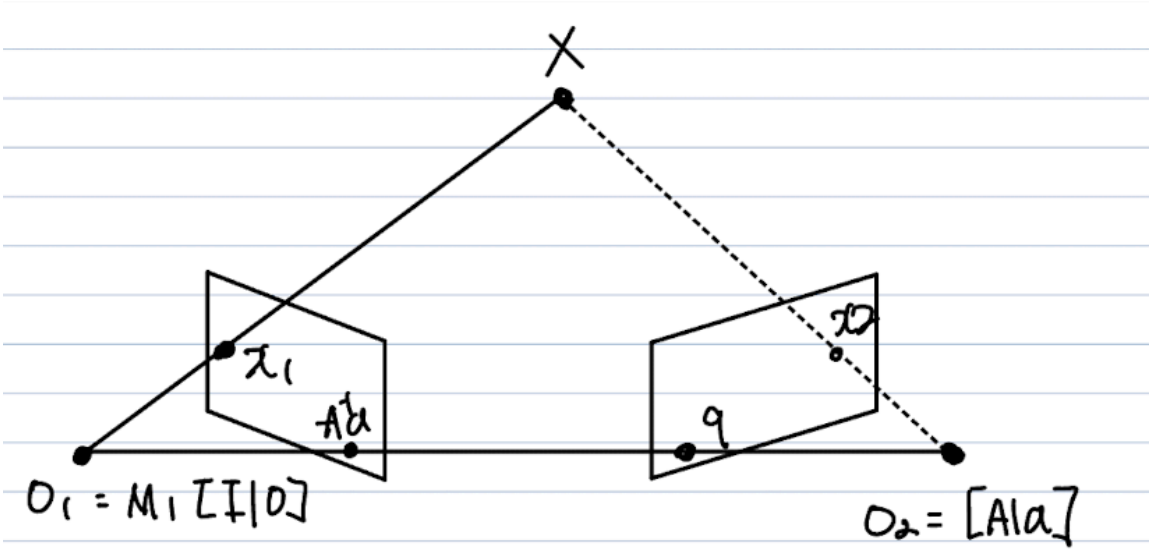


(1) In lab8 (instead of lab5, which I think this was a typo) optical flow, the goal is to estimate the motion of pixels between two consecutive frames or two different views by minimizing a photometric loss over a 2D neighborhood. However, when the epipolar geometry is known, we can significantly reduce the search space using the fundamental matrix F, which encodes the geometric relationship between two camera views. Specifically, for a point x in the first image, its corresponding point x_2 in the second image must lie on the epipolar line $l_2 = Fx_1$. Therefore, instead of searching in a 2D window around x_2, we can restrict the search along this epipolar line, which is a 1D constraint. This greatly improves the accuracy and efficiency of optical flow estimation in stereo or multi-view settings. To incorporate this constraint into the Lab 5 optical flow implementation, one would compute the epipolar line for each pixel in the first image using the fundamental matrix and constrain the matching cost calculation to points along this line in the second image.

(2) Given the two camera projection matrices $M_1 = [I \mid 0]$ and $M_2 = [A \mid a]$, where A is a nonsingular 3×3 matrix and a is a translation vector, we aim to prove that the vector a corresponds to an epipole. The epipole in image 2 is defined as the projection of the first camera center C_1 into the second image. Since M_1 is $[I \mid 0]$, the first camera center in homogeneous coordinates is $C_1 = [0, 0, 0, 1]^T$. Projecting this point using M_2 gives:

$$e_2 = M_2 C_1 = [A \mid a] \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = a$$

Thus, the epipole in image 2 is precisely the translation vector a, confirming that a is an epipole. Geometrically, this makes sense: the epipole in an image is the projection of the other camera's center, and since the second camera is located at a with respect to the first, it projects to a in image 2. In a diagram, this would be represented by rays from both camera centers intersecting at the 3D point, with the epipole lying at the intersection of these rays projected into each image plane.



6. **3D Estimation [Extra credit - 10 pts bonus].** Design a bundle adjuster that allows for arbitrary chains of transformations and prior knowledge about the unknowns, see SZ Figures 11.14-11.15 for an example.

7. **Vanishing points [12 pts total]** Using `ps5_example.jpg`, you need to estimate the three major orthogonal vanishing points. Use at least three manually selected lines to solve for each vanishing point. The starter code below provides an interface for selecting and drawing the lines, but the code for computing the vanishing point needs to be inserted.

In [5]:
```
%matplotlib tk
import matplotlib.pyplot as plt
import numpy as np

from PIL import Image
```

```
def get_input_lines(im, min_lines=3): # when the user click two ends points
    """
    Allows user to input line segments; computes centers and directions.
    Inputs:
        im: np.ndarray of shape (height, width, 3)
        min_lines: minimum number of lines required
    Returns:
        n: number of lines from input
        lines: np.ndarray of shape (3, n)
            where each column denotes the parameters of the line equation
        centers: np.ndarray of shape (3, n)
            where each column denotes the homogeneous coordinates of the cer
    """
    n = 0
    lines = np.zeros((3, 0))
    centers = np.zeros((3, 0))

    plt.figure()
    plt.axis('off')
    plt.imshow(im)
    print(f'Set at least {min_lines} lines to compute vanishing point')
    print(f'The delete and backspace keys act like right clicking')
    print(f'The enter key acts like middle clicking')
    while True:
        print('Click the two endpoints, use the right button (delete and bac
        clicked = plt.ginput(2, timeout=0, show_clicks=True)
        if not clicked or len(clicked) < 2:
            if n < min_lines:
                print(f'Need at least {min_lines} lines, you have {n} now')
                continue
            else:
                # Stop getting lines if number of lines is enough
                break

        # Unpack user inputs and save as homogeneous coordinates
        pt1 = np.array([clicked[0][0], clicked[0][1], 1])
        pt2 = np.array([clicked[1][0], clicked[1][1], 1])
        # Get line equation using cross product
        # Line equation: line[0] * x + line[1] * y + line[2] = 0
        line = np.cross(pt1, pt2)
        lines = np.append(lines, line.reshape((3, 1)), axis=1)
        # Get center coordinate of the line segment
        center = (pt1 + pt2) / 2
        centers = np.append(centers, center.reshape((3, 1)), axis=1)

        # Plot line segment
        plt.plot([pt1[0], pt2[0]], [pt1[1], pt2[1]], color='b')

        n += 1

    return n, lines, centers

def plot_lines_and_vp(ax, im, lines, vp): # draw lines and calculated vanish
    """
    Plots user-input lines and the calculated vanishing point.
    Inputs:
```

```
        im: np.ndarray of shape (height, width, 3)
        lines: np.ndarray of shape (3, n)
            where each column denotes the parameters of the line equation
        vp: np.ndarray of shape (3, )
    """
    bx1 = min(1, vp[0] / vp[2]) - 10
    bx2 = max(im.shape[1], vp[0] / vp[2]) + 10
    by1 = min(1, vp[1] / vp[2]) - 10
    by2 = max(im.shape[0], vp[1] / vp[2]) + 10

    ax.imshow(im)
    for i in range(lines.shape[1]):
        if lines[0, i] < lines[1, i]:
            pt1 = np.cross(np.array([1, 0, -bx1]), lines[:, i])
            pt2 = np.cross(np.array([1, 0, -bx2]), lines[:, i])
        else:
            pt1 = np.cross(np.array([0, 1, -by1]), lines[:, i])
            pt2 = np.cross(np.array([0, 1, -by2]), lines[:, i])
        pt1 = pt1 / pt1[2]
        pt2 = pt2 / pt2[2]
        ax.plot([pt1[0], pt2[0]], [pt1[1], pt2[1]], 'g')

    ax.plot(vp[0] / vp[2], vp[1] / vp[2], 'ro')
    ax.set_xlim([bx1, bx2])
    ax.set_ylim([by2, by1])

def get_top_and_bottom_coordinates(im, obj): # get object's top and bottom c
    """
    For a specific object, prompts user to record the top coordinate and the
    Inputs:
        im: np.ndarray of shape (height, width, 3)
        obj: string, object name
    Returns:
        coord: np.ndarray of shape (3, 2)
            where coord[:, 0] is the homogeneous coordinate of the top of th
            coordinate of the bottom
    """
    plt.figure()
    plt.imshow(im)

    print('Click on the top coordinate of %s' % obj)
    clicked = plt.ginput(1, timeout=0, show_clicks=True)
    x1, y1 = clicked[0]
    # Uncomment this line to enable a vertical line to help align the two co
    # plt.plot([x1, x1], [0, im.shape[0]], 'b')
    print('Click on the bottom coordinate of %s' % obj)
    clicked = plt.ginput(1, timeout=0, show_clicks=True)
    x2, y2 = clicked[0]

    plt.plot([x1, x2], [y1, y2], 'b')

    return np.array([[x1, x2], [y1, y2], [1, 1]])
```

7.1. **Estimating Horizon [3 pts]** You should: a) plot the VPs and the lines used to estimate the vanishing points (VP) on the image plane using the provided code. b)

Specify the VP pixel coordinates. c) Plot the ground horizon line and specify its parameters in the form $a * x + b * y + c = 0$. Normalize the parameters so that: $a^2 + b^2 = 1$.

In [7]:
```
def get_vanishing_point(lines):
    """
    Solves for the vanishing point using the user-input lines.
    """
    # <YOUR CODE>
    A = lines.T
    _, _, Vt = np.linalg.svd(A)
    vp = Vt[-1]
    return vp / vp[2]

def get_horizon_line(vp1, vp2):
    """
    Calculates the ground horizon line.
    """
    # <YOUR IMPLEMENTATION>
    line = np.cross(vp1, vp2)
    line_norm = line / np.linalg.norm(line[:2])
    return line_norm

def plot_horizon_line(im, horizon_line):
    """
    Plots the horizon line.
    """
    # <YOUR IMPLEMENTATION>
    a, b, c = horizon_line
    fig, ax = plt.subplots()
    ax.imshow(im)

    x_vals = np.linspace(0, im.shape[1], 500)
    y_vals = -(a * x_vals + c) / b

    ax.plot(x_vals, y_vals, 'r--', label="Horizon Line")
    ax.set_xlim([0, im.shape[1]])
    ax.set_ylim([im.shape[0], 0])
    ax.legend()

    plt.title("Estimate Horizon Line")
    plt.show()

    print("Horizon line equation (normalized):")
    print(f"{a:.4f} * x + {b:.4f} * y + {c:.4f} = 0")
    print(f"→ where a² + b² = {a**2 + b**2:.4f}")

im = np.asarray(Image.open('ps5_example.jpg'))

# Get vanishing points for each of the directions
num_vpts = 3
vpts = np.zeros((3, num_vpts)) # 3 x 3

fig, axs = plt.subplots(1, 3, figsize=(18, 6))
all_lines = []
```

```
for i in range(num_vpts):
    print('Getting vanishing point %d' % i)
    # Get at least three lines from user input
    n, lines, centers = get_input_lines(im) # select 3 points for each vanis
    # <YOUR IMPLEMENTATION> Solve for vanishing point
    vpts[:, i] = get_vanishing_point(lines)
    all_lines.append(lines)
    # Plot the lines and the vanishing point
    plot_lines_and_vp(axs[i], im, lines, vpts[:, i])

# Show all 3 vanishing points

fig.tight_layout()
fig.suptitle('Vanishing Points and Estimating Lines')
plt.show()

# <YOUR IMPLEMENTATION> Get the ground horizon line
horizon_line = get_horizon_line(vpts[:, 0], vpts[:, 1])
# <YOUR IMPLEMENTATION> Plot the ground horizon line
plot_horizon_line(im, horizon_line)
```

```
Getting vanishing point 0
Set at least 3 lines to compute vanishing point
The delete and backspace keys act like right clicking
The enter key acts like middle clicking
Click the two endpoints, use the right button (delete and backspace keys) to
undo, and use the middle button to stop input
Click the two endpoints, use the right button (delete and backspace keys) to
undo, and use the middle button to stop input
Click the two endpoints, use the right button (delete and backspace keys) to
undo, and use the middle button to stop input
Click the two endpoints, use the right button (delete and backspace keys) to
undo, and use the middle button to stop input
Getting vanishing point 1
Set at least 3 lines to compute vanishing point
The delete and backspace keys act like right clicking
The enter key acts like middle clicking
Click the two endpoints, use the right button (delete and backspace keys) to
undo, and use the middle button to stop input
Click the two endpoints, use the right button (delete and backspace keys) to
undo, and use the middle button to stop input
Click the two endpoints, use the right button (delete and backspace keys) to
undo, and use the middle button to stop input
Click the two endpoints, use the right button (delete and backspace keys) to
undo, and use the middle button to stop input
Getting vanishing point 2
Set at least 3 lines to compute vanishing point
The delete and backspace keys act like right clicking
The enter key acts like middle clicking
Click the two endpoints, use the right button (delete and backspace keys) to
undo, and use the middle button to stop input
Click the two endpoints, use the right button (delete and backspace keys) to
undo, and use the middle button to stop input
Click the two endpoints, use the right button (delete and backspace keys) to
undo, and use the middle button to stop input
Click the two endpoints, use the right button (delete and backspace keys) to
undo, and use the middle button to stop input
Horizon line equation (normalized):
-0.0160 * x + 0.9999 * y + -211.0833 = 0
→ where a² + b² = 1.0000
```

In [8]:
```
for i in range(3):
  vp = vpts[:, i] / vpts[2, i]
  print(vp)
```

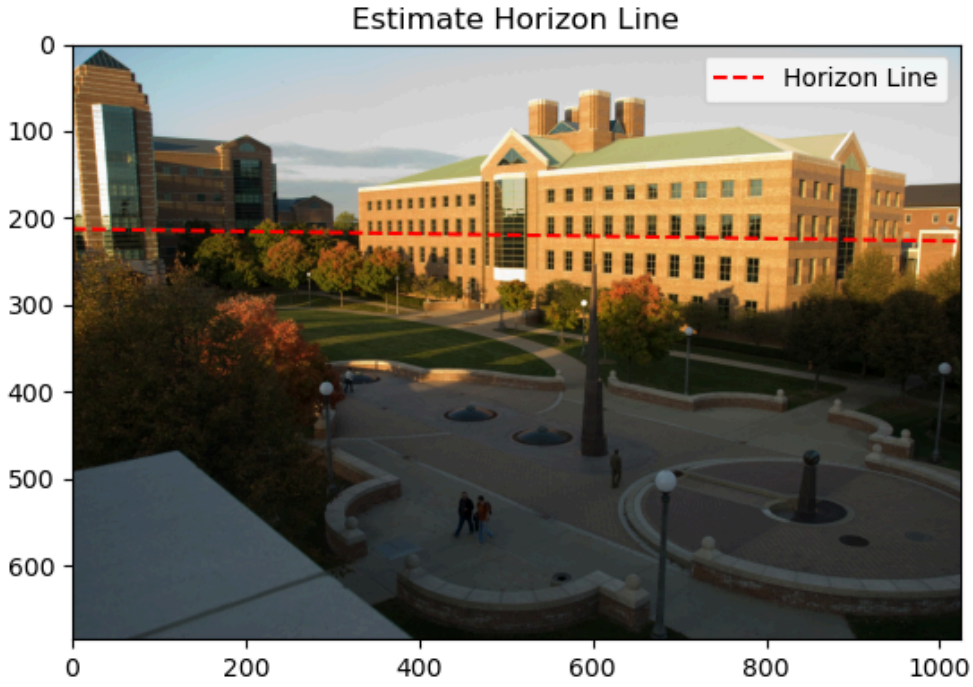
```
[-208.03787413  207.78548663    1.        ]
[1.41334928e+03 2.33697972e+02 1.00000000e+00]
[5.55905443e+02 5.86056984e+03 1.00000000e+00]
```

Horizon Line Equation: -0.0160 * x + 0.9999 * y + -211.0833 = 0

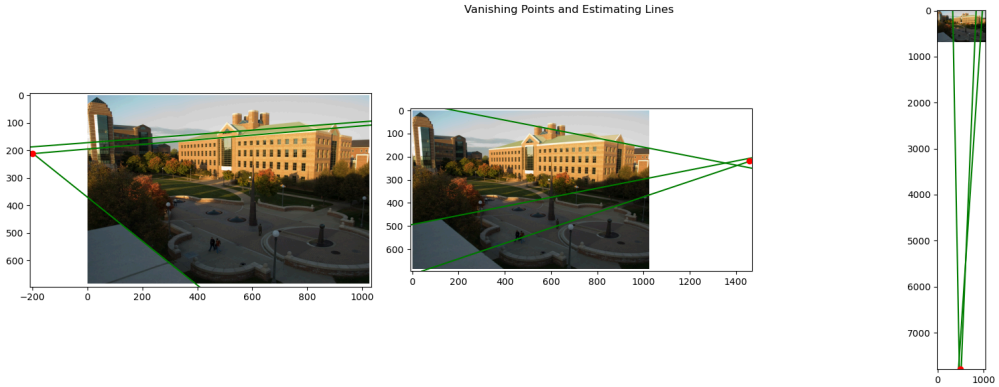
Vanishing Points pixels Coordinates

Vanishing Points pixels Coordinates [-198.52921804 210.8593942 1.] [1.45919861e+03 2.17614732e+02 1.00000000e+00] [4.91325640e+02 7.78642205e+03 1.00000000e+00]

Ground Horizion Line



Vanishing Points



7.2. **Solving for camera parameters [3 pts]** Using the fact that the vanishing directions are orthogonal, solve for the focal length and optical center (principal point) of the camera. Show all your work and include the computed parameters in your report.

In [12]:
```
import sympy as sp

def get_camera_parameters(vpts):
    """
    Computes the camera parameters. Hint: The SymPy package is suitable for
    """
    # <YOUR CODE>
    import numpy as np
```

```
    f, u, v = sp.symbols('f u v', real=True)


    vpts = np.array(vpts)



    x1, y1 = vpts[0, 0], vpts[1, 0]
    x2, y2 = vpts[0, 1], vpts[1, 1]
    x3, y3 = vpts[0, 2], vpts[1, 2]

    eq1 = (x1 - u)*(x2 - u) + (y1 - v)*(y2 - v) + f**2
    eq2 = (x2 - u)*(x3 - u) + (y2 - v)*(y3 - v) + f**2
    eq3 = (x3 - u)*(x1 - u) + (y3 - v)*(y1 - v) + f**2

    # Solve symbolically
    sol = sp.solve([eq1, eq2, eq3], (f, u, v), dict=True)

    # Pick a valid solution: real and positive focal length
    for s in sol:
        f_val = s[f]
        if sp.im(f_val) == 0 and f_val > 0:
            return float(f_val), float(s[u]), float(s[v])

    raise ValueError("No valid solution found. Try new vanishing Points")

# <YOUR IMPLEMENTATION> Solve for the camera parameters (f, u, v)
f, u, v = get_camera_parameters(vpts)
print(f"Estimated focal length 'f' : {f:.2f}")
print(f"Principal point: (u, v) = ({u:.2f}, {v:.2f})")
```

```
Estimated focal length 'f' : 801.25
Principal point: (u, v) = (644.17, 337.65)
```

Computed Parameters

Estimated focal length: 816.93

Principal point: (u, v) = (521.82, 302.98)

7.3. **Camera rotation matrix [3 pts]** Compute the rotation matrix for the camera, setting the vertical vanishing point as the Y-direction, the right-most vanishing point as the X-direction, and the left-most vanishing point as the Z-direction.

In [13]:
```
def get_k(f, u, v):
    K = np.array([
        [f, 0, u],
        [0, f, v],
        [0, 0, 1]
    ])

    return K

def get_rotation_matrix(k, vpts):
```

```
    """
    Computes the rotation matrix using the camera parameters.
    """
    # <YOUR CODE>
    K_inverse = np.linalg.inv(k)

    # Step 3: Back-project vanishing points
    # vx = vpts[:, 1]  # X-direction (right)
    # vy = vpts[:, 0]  # Y-direction (vertical)
    # vz = vpts[:, 2]  # Z-direction (left)

    vx = vpts[:, 1]  # X-direction (right)
    vy = vpts[:, 2]  # Y-direction (vertical)
    vz = vpts[:, 0]  # Z-direction (left)


    dx = K_inverse @ vx; dx /= np.linalg.norm(dx)
    dy = K_inverse @ vy; dy /= np.linalg.norm(dy)
    dz = K_inverse @ vz; dz /= np.linalg.norm(dz)

    # Step 4: Compose rotation matrix
    R = np.stack((dx, dy, dz), axis=1)  # [dx | dy | dz]

    return R

# <YOUR IMPLEMENTATION> Solve for the rotation matrix
K = get_k(f, u, v)
R = get_rotation_matrix(K, vpts)
print(f"Rotation matrix:\n", R)
```

```
Rotation matrix:
 [[ 0.68950855 -0.01581412 -0.72410488]
 [-0.09318336  0.98951581 -0.11034182]
 [ 0.71825819  0.14355615  0.680806  ]]
```

7.4. **Measurement estimation [3 pts]** Estimate the heights of (a) the large building in the center of the image, (b) the spike statue, and (c) the lamp posts assuming that the person nearest to the spike is 5ft 6in tall. In the report, show all the lines and measurements used to perform the calculation. How do the answers change if you assume the person is 6ft tall?

In [14]:
```
def estimate_height(horizon_line, target, reference, ref_height):
    """
    Estimates height for a specific object using the recorded coordinates. Y
    your report.
    """
    # <YOUR IMPLEMENTATION>
    target_t, target_b = target[:, 0], target[:, 1]
    reference_t, reference_b = reference[:, 0], reference[:, 1]

    # Compute intersection point r: r = target_t × horizon_line
    r = np.cross(target_t, horizon_line)
    r = r / r[2]

    # Convert all points to inhomogeneous (2D)
```

```
    b_tar_2d = target_b[:2] / target_b[2]
    r_2d = r[:2] / r[2]
    b_ref_2d = reference_b[:2] / reference_b[2]
    t_ref_2d = reference_t[:2] / reference_t[2]

    # # Compute lengths for cross ratio
    def dist(a, b):
        return np.linalg.norm(a - b)

    target_ratio = dist(b_tar_2d, r_2d)
    reference_ratio = dist(b_ref_2d, t_ref_2d)

    height = ref_height * (target_ratio / reference_ratio)

    return height

# Record image coordinates for each object and store in map
objects = ('person', 'Building', 'the spike statue', 'the lamp posts')
coords = dict()
for obj in objects:
    coords[obj] = get_top_and_bottom_coordinates(im, obj)

reference_obj = 'person'
ref_height = 5.6
# vp_y = vpts[:, 2] # I chose Last one as the y vanishing point

# <YOUR IMPLEMENTATION> Estimate heights
for obj in objects[1:]:
    height = estimate_height(horizon_line=horizon_line, target=coords[obj],
    print('Estimating height of %s : %.2f' % (obj, height) )
```

```
Click on the top coordinate of person
Click on the bottom coordinate of person
Click on the top coordinate of Building
Click on the bottom coordinate of Building
Click on the top coordinate of the spike statue
Click on the bottom coordinate of the spike statue
Click on the top coordinate of the lamp posts
Click on the bottom coordinate of the lamp posts
Estimating height of Building : 86.16
Estimating height of the spike statue : 99.41
Estimating height of the lamp posts : 131.30
```

In [16]:
```
def estimate_height(vp_y, target, reference, ref_height):
    """
    Estimates height for a specific object using the recorded coordinates. Y
    your report.
    """
    # <YOUR IMPLEMENTATION>
    target_t, target_b = target[:, 0], target[:, 1]
    reference_t, reference_b = reference[:, 0], reference[:, 1]

    target_t, target_b = target_t[:2], target_b[:2]
    reference_t, reference_b = reference_t[:2], reference_b[:2]

    vp_y = vp_y[:2]
```

```
    # Compute lengths for cross ratio
    def dist(a, b):
        return np.linalg.norm(a - b)

    cr_tar = dist(vp_y, target_b) / dist(vp_y, target_t)
    cr_ref = dist(vp_y, reference_b) / dist(vp_y, reference_t)

    height_target = ref_height * (cr_tar / cr_ref)

    return height_target

# Record image coordinates for each object and store in map
objects = ('person', 'Building', 'the spike statue', 'the lamp posts')
coords = dict()
for obj in objects:
    coords[obj] = get_top_and_bottom_coordinates(im, obj)

reference_obj = 'person'
ref_height = 5.6
vp_y = vpts[:, 2] # I chose Last one as the y vanishing point

# <YOUR IMPLEMENTATION> Estimate heights
for obj in objects[1:]:
    height = estimate_height(vp_y=vp_y, target=coords[obj], reference=coords
    print('Estimating height of %s : %.2f' % (obj, height) )
    # print(f'{height}')
```

```
Click on the top coordinate of person
Click on the bottom coordinate of person
Click on the top coordinate of Building
Click on the bottom coordinate of Building
Click on the top coordinate of the spike statue
Click on the bottom coordinate of the spike statue
Click on the top coordinate of the lamp posts
Click on the bottom coordinate of the lamp posts
Estimating height of Building : 5.40
Estimating height of the spike statue : 5.36
Estimating height of the lamp posts : 5.45
```

8. **Warped view [5 bonus pts]** Compute and display rectified views of the ground plane and the large building in the center of the image.

Checklist for PS4 **Report**: (Please include them in your report to get the corresponding credits)

1. Q1: **- (2 pts)**

- printed lab 1 camera projection (3 x 4 matrix) **- (0.5 pts)**
- printed lab 2 camera projection (3 x 4 matrix) **- (0.5 pts)**

- residual in lab 1 **- (0.5 pts)**
- residual in lab 2 **- (0.5 pts)**

2. Q2: **- (2 pts)**

- printed lab1 camera center **- (1 pts)**
- printed lab2 camera center **- (1 pts)**

3. Q3: **- (2 pts)**

- a 3D display of the two camera centers and the reconstructed points. **- (1 pts)**
- the residuals between the observed 2D points and the projected 3D poins in the two images. **- (1 pts)**

4. Q4 (BONUS): **- (2 pts)**

- number of inliers **- (1 pts)**
- average residual for the inliers **- (1 pts)**

5. Q7 **- (8 pts)**

- 7.1 A ps5_example.jpg with plotted vanishing points and lines to estimate the VPs. **- (2 pts)**
- 7.2 The focal length and optical center (principal point) of the camera. **- (2 pts)**
- 7.3 The rotation matrix for the camera **- (2 pts)**
- 7.4 The estimated heights of the large building in the center of the image, the spike statue, and the lamp posts . **- (1 pts)**
- 7.4 A brief explanation of the calculation process, i.e. show all the lines and measurements used to perform the calculation. **- (1 pts)**

In [ ]: